

JavaScript ES Modules – Tasks 1 to 14

Complete solutions covering named exports, default exports, imports, utility refactoring, and a short scalability explanation. Tasks that are repeated in the question list are included separately.

Task 1. Generate and export an order ID

Create `utils.js` with `generateOrderId()` and use it from `order.js`.

`utils.js`

```
export function generateOrderId() {
  const characters =
    'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';

  let orderId = '';

  for (let i = 0; i < 8; i++) {
    orderId += characters[
      Math.floor(Math.random() * characters.length)
    ];
  }

  return orderId;
}
```

`order.js`

```
import { generateOrderId } from './utils.js';

console.log('New Order ID:', generateOrderId());
```

Task 2. Generate and export an order ID (same requirement as Task 1)

This is a duplicate of Task 1, so the same implementation can be used.

`utils.js`

```
export function generateOrderId() {
  const characters = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';
  let orderId = '';

  for (let i = 0; i < 8; i++) {
    orderId += characters[Math.floor(Math.random() * characters.length)];
  }

  return orderId;
}
```

`order.js`

```
import { generateOrderId } from './utils.js';

console.log('New Order ID:', generateOrderId());
```

Task 3. Format prices and calculate discounted prices

Create two named exports in `utils.js`.

`utils.js`

```
export function formatPrice(price) {
```

```

    return `■${price.toFixed(2)}`;
  }

  export function getDiscountedPrice(price, discount) {
    return price - (price * discount) / 100;
  }

```

Task 4. Spotify utilities

`formatSongTitle` capitalizes every word, while `getSongDurationInMinutes` converts seconds to mm:ss. Both are named exports.

spotifyUtils.js

```

export function formatSongTitle(title) {
  return title
    .split(' ')
    .map(word => word.charAt(0).toUpperCase() + word.slice(1).toLowerCase())
    .join(' ');
}

export function getSongDurationInMinutes(seconds) {
  const minutes = Math.floor(seconds / 60);
  const remainingSeconds = seconds % 60;

  return `${String(minutes).padStart(2, '0')}:${String(remainingSeconds).padStart(2, '0')}`;
}

```

main.js

```

import {
  formatSongTitle,
  getSongDurationInMinutes
} from './spotifyUtils.js';

const songTitle = 'blinding lights';
const duration = 203;

console.log('Song:', formatSongTitle(songTitle));
console.log('Duration:', getSongDurationInMinutes(duration));

```

Task 5. Spotify utilities (same requirement as Task 4)

This is a duplicate of Task 4, so the same implementation can be used.

spotifyUtils.js

```

export function formatSongTitle(title) {
  return title
    .split(' ')
    .map(word => word.charAt(0).toUpperCase() + word.slice(1).toLowerCase())
    .join(' ');
}

export function getSongDurationInMinutes(seconds) {
  const minutes = Math.floor(seconds / 60);
  const remainingSeconds = seconds % 60;

  return `${String(minutes).padStart(2, '0')}:${String(remainingSeconds).padStart(2, '0')}`;
}

```

main.js

```

import {
  formatSongTitle,
  getSongDurationInMinutes
} from './spotifyUtils.js';

```

```
const songTitle = 'blinding lights';
const duration = 203;

console.log('Song:', formatSongTitle(songTitle));
console.log('Duration:', getSongDurationInMinutes(duration));
```

Task 6. Use price utilities in cart.js

Import both named functions from utils.js and display original and discounted prices for three items.

utils.js

```
export function formatPrice(price) {
  return `■${price.toFixed(2)}`;
}

export function getDiscountedPrice(price, discount) {
  return price - (price * discount) / 100;
}
```

cart.js

```
import {
  formatPrice,
  getDiscountedPrice
} from './utils.js';

const items = [
  { name: 'T-Shirt', price: 799, discount: 10 },
  { name: 'Shoes', price: 2499, discount: 15 },
  { name: 'Backpack', price: 1499, discount: 20 }
];

items.forEach(item => {
  const discountedPrice = getDiscountedPrice(
    item.price,
    item.discount
  );

  console.log(item.name);
  console.log('Original:', formatPrice(item.price));
  console.log('Discounted:', formatPrice(discountedPrice));
});
```

Task 7. Default export for Instagram follower formatting

Declare formatFollowers as a default export and import it into profile.js.

instaHelpers.js

```
export default function formatFollowers(count) {
  if (count >= 1_000_000) {
    return `${(count / 1_000_000).toFixed(1)}M`;
  }

  if (count >= 1_000) {
    return `${(count / 1_000).toFixed(1)}K`;
  }

  return String(count);
}
```

profile.js

```
import formatFollowers from './instaHelpers.js';

const users = [
```

```

    { name: 'Aarav', followers: 1200 },
    { name: 'Meera', followers: 2500000 },
    { name: 'Riya', followers: 850 }
  ];

  users.forEach(user => {
    console.log(
      `${user.name}: ${formatFollowers(user.followers)} followers`
    );
  });

```

Task 8. Default export for Instagram follower formatting (same requirement as Task 7)

This is a duplicate of Task 7, so the same implementation can be used.

instaHelpers.js

```

export default function formatFollowers(count) {
  if (count >= 1_000_000) {
    return `${(count / 1_000_000).toFixed(1)}M`;
  }

  if (count >= 1_000) {
    return `${(count / 1_000).toFixed(1)}K`;
  }

  return String(count);
}

```

profile.js

```

import formatFollowers from './instaHelpers.js';

const users = [
  { name: 'Aarav', followers: 1200 },
  { name: 'Meera', followers: 2500000 },
  { name: 'Riya', followers: 850 }
];

users.forEach(user => {
  console.log(
    `${user.name}: ${formatFollowers(user.followers)} followers`
  );
});

```

Task 9. Refactor getDiscountedPrice as a default export

Change utils.js so getDiscountedPrice is the default export, while formatPrice remains a named export.

utils.js

```

export function formatPrice(price) {
  return `■${price.toFixed(2)}`;
}

export default function getDiscountedPrice(price, discount) {
  return price - (price * discount) / 100;
}

```

cart.js

```

import getDiscountedPrice, {
  formatPrice
} from './utils.js';

```

```

const price = 2000;
const discount = 15;

const discountedPrice = getDiscountedPrice(price, discount);

console.log('Original:', formatPrice(price));
console.log('Discounted:', formatPrice(discountedPrice));

```

Task 10. Refactor Flipkart-style cart functions

Move `addToCart`, `removeFromCart`, and `calculateTotal` into `cartUtils.js` as named exports, then import them into `main.js`.

cartUtils.js

```

export function addToCart(cart, item) {
  cart.push(item);
  return cart;
}

export function removeFromCart(cart, productId) {
  return cart.filter(item => item.id !== productId);
}

export function calculateTotal(cart) {
  return cart.reduce((total, item) => total + item.price, 0);
}

```

main.js

```

import {
  addToCart,
  removeFromCart,
  calculateTotal
} from './cartUtils.js';

let cart = [];

cart = addToCart(cart, {
  id: 1,
  name: 'Phone',
  price: 15000
});

cart = addToCart(cart, {
  id: 2,
  name: 'Headphones',
  price: 2500
});

console.log('Cart:', cart);
console.log('Total:', calculateTotal(cart));

cart = removeFromCart(cart, 1);

console.log('After removal:', cart);
console.log('New total:', calculateTotal(cart));

```

Task 11. Refactor Flipkart-style cart functions (same requirement as Task 10)

This is a duplicate of Task 10, so the same implementation can be used.

cartUtils.js

```

export function addToCart(cart, item) {

```

```

    cart.push(item);
    return cart;
}

export function removeFromCart(cart, productId) {
    return cart.filter(item => item.id !== productId);
}

export function calculateTotal(cart) {
    return cart.reduce((total, item) => total + item.price, 0);
}

```

main.js

```

import {
    addToCart,
    removeFromCart,
    calculateTotal
} from './cartUtils.js';

let cart = [];

cart = addToCart(cart, { id: 1, name: 'Phone', price: 15000 });
cart = addToCart(cart, { id: 2, name: 'Headphones', price: 2500 });

console.log('Total:', calculateTotal(cart));

cart = removeFromCart(cart, 1);

console.log('After removal:', cart);
console.log('New total:', calculateTotal(cart));

```

Task 12. Refactor Flipkart-style wishlist functions

Move wishlist add/remove/list functions into wishlistUtils.js and import them back into wishlist.js using named imports.

wishlistUtils.js

```

export function addToWishlist(wishlist, item) {
    wishlist.push(item);
    return wishlist;
}

export function removeFromWishlist(wishlist, productId) {
    return wishlist.filter(item => item.id !== productId);
}

export function listWishlistItems(wishlist) {
    return wishlist;
}

```

wishlist.js

```

import {
    addToWishlist,
    removeFromWishlist,
    listWishlistItems
} from './wishlistUtils.js';

let wishlist = [];

wishlist = addToWishlist(wishlist, {
    id: 1,
    name: 'Smartphone'
});

wishlist = addToWishlist(wishlist, {

```

```
    id: 2,  
    name: 'Smartwatch'  
  });  
  
  console.log('Wishlist:', listWishlistItems(wishlist));  
  
  wishlist = removeFromWishlist(wishlist, 1);  
  
  console.log('Updated wishlist:', listWishlistItems(wishlist));
```

Task 13. How imports and exports help a Zomato-like app scale

Answer in 3–4 sentences with one concrete example.

Answer

Splitting utility functions into separate files keeps each feature focused and makes the codebase easier to maintain.

Task 14. How imports and exports help a Zomato-like app scale (same requirement as Task 13)

This is a duplicate of Task 13, so the same explanation can be used.

Answer

Splitting utility functions into separate files keeps each feature focused and makes the codebase easier to maintain.

Note: The examples use ES modules, so in a Node.js project you should configure the project for ES modules (for example, by using "type": "module" in package.json) or use an environment that supports module syntax.